DOC: **LR2-REDUX / REVISION NOTES** REV. **1.0**

SUBJECT BOARD: **LITTER-ROBOT REV.2C** (2003, INTENSA INC.)

STATUS: **BENCH-VERIFIED, ASSEMBLY IN PROGRESS**

Rebuilding a dead Litter-Robot 2 control board

The original control board died — not the sensors, not the switches, not the motor. Instead of hunting for a 20-year-old replacement for a dead PIC microcontroller, this project replaces the whole control board with an ESP32 and reuses every other stock part exactly as it was: the gearmotor, the hall sensors, the weight switch, the anti-pinch switch, the wiring harness itself. This page is the story of how that went, in the order it actually happened — including the parts that didn't work the first time.

HARNESS PINOUT **Fully resolved**	HALL SENSORS **Installed in chassis**	WEIGHT / ANTI-PINCH SPLIT **Done, bench-verified**	FIRMWARE **Builds clean, both targets**
MOTOR / FULL ASSEMBLY In progress	REAL-HARDWARE CYCLE TIMING **Placeholder values**		

01 The problem

The stock board is stamped **"LITTER-ROBOT Rev.2C, Copyright (C) 2003 Intensa Inc."** — a PIC16C622A-based controller that simply died. Not the Home/Dump hall-effect position sensors, not the weight or anti-pinch switches, not the gearmotor, not the harness connecting them all — those were all still good. The brain was gone.

The original chip is old, unsupported, and effectively irreplaceable without reverse-engineering its firmware from scratch. Rather than try to source or clone a 20-year-old PIC controller, the

plan became: replace the whole control board with an ESP32 and an L298N H-bridge, and wire every still-good stock part into it exactly as it was — the 12V gearmotor, the hall sensors and their existing magnets on the globe, the cat-weight switch, the anti-pinch switch, the wiring harness itself.

02 Reading the corpse

Before any of the new electronics could go in, the stock harness's actual wiring had to be established — not assumed. Early attempts to find a pinout online turned up two AI-generated summaries that contradicted each other (one literally hedged *"Pin 1: Positive/Negative"*) and didn't match anything verifiable. Those were discarded outright.

What actually resolved it was a direct multimeter continuity trace across both connectors on the physical board, cross-checked against a genuine third-party teardown of the same board family (a 2020 student reverse-engineering writeup that turned out to describe the exact same connector, down to the wire colors).

CONNECTOR	CARRIES	RESOLVED VIA
4-pin	Motor (to driver IC "U5")	Continuity trace
7-pin	Hall sensors + weight/anti-pinch switches	Continuity trace + bench test

Opening the hall sensor package itself settled the last unknown: two **Allegro A1101EUA-T** hall-effect ICs, mounted facing each other, open-drain outputs pulled up through a 330Ω network on the board. A part number, not a guess.

03 The series-wiring trap

Pins 6 and 7 of the 7-pin connector looked, at first, like two independent switch inputs: the weight switch and the anti-pinch switch, each presumably grounded on its own. Wiring pin 6 alone to a pulled-up GPIO read *nothing*.

Tying pin 7 to ground made pin 6 start working — which meant the two switches weren't independent at all. They were wired in **series**, on one shared loop, with the original PIC almost certainly using one of its own pins as a firmware-driven "virtual ground" for the whole loop rather than true board ground.

> WHY THIS MATTERS, NOT JUST A WIRING CURIOSITY
>
> Cycling only ever starts after the cat has already left — the weight switch is already open by the time the globe moves. On a shared series loop, a mid-cycle pinch produces **zero observable change** on that one signal. It's not an ambiguous read; a truth table proves no amount of clever firmware can tell "cat gone, cycling normally" apart from "cat stuck under the globe" on a single combined wire.

The same failure mode, independently, in someone else's teardown of this exact board family:

> INDEPENDENTLY CONFIRMED, 2020 TEARDOWN OF THE SAME BOARD
>
> *"...totally stupid, [it makes it] impossible to distinguish a cat in the litter from a cat stuck under the globe."*

Motor current/stall sensing was considered as a software-only workaround and rejected — too failure-prone on an old, uncalibrated gearbox without real bench work to back it up. The fix that shipped was physical: open the case, find the internal jumper joining the two switches, cut it, and give each switch its own independent ground return. See §05 for how that actually went.

04 Design decisions

A few choices made early on shaped everything downstream:

- **WiFi/MQTT are runtime config, not compile-time secrets.** Set once through a captive setup portal and stored in NVS — the same firmware image works on any unit, no per-device rebuild.
- **The browser never speaks MQTT.** The dashboard was originally going to connect to the MQTT broker directly (MQTT-over-WebSockets), but that needed a broker websocket listener and pulled in polyfills for no real benefit. It talks to a plain WebSocket server running on the ESP32 itself instead — simpler network setup, zero broker dependency for the dashboard.
- **Setup mode is reboot-based.** Hot-swapping between "AP + captive portal" and "STA + dashboard" inside one running process is fiddly with two async servers. Instead: flag it in NVS, restart, let boot branch cleanly. Boring, and it works.
- **Home and Dump are defined by GPIO, not by which stock sensor is which.** The original board's "sensor A / sensor B" identity was never load-bearing for a rebuild — whichever physical sensor ends up mounted at each position just wires to that position's GPIO.

05 Bring-up, and the harness split

Before trusting the real state machine with a live gearmotor, a standalone diagnostic firmware image came first — its own SoftAP, no WiFi credentials or MQTT needed, one page showing live raw and debounced state for every sensor input plus manual motor jog controls (with safety interlocks: auto-stop on a tripped weight switch, a dropped browser tab, or a lost WebSocket). It's what actually proved the series-wiring theory from §03 on the bench, and what verified the fix afterward.

The fix itself: open the case, locate the jumper wire bridging the weight and anti-pinch switches, cut it, run a new ground return to each. The harness's outward wiring didn't need to change — the same two conductors that always went to the connector now simply carry two independent signals instead of one shared loop.

Fig. 1 — internal jumper cut, independent ground returns wired to each switch.

Bench-verified afterward with all three test cases — weight alone, anti-pinch alone, both together — each reading independently on the diagnostic tool, exactly as the truth table

predicted they should.

> A SECOND BUG, CAUGHT THE SAME DAY
>
> The anti-pinch switch turned out to be **normally-closed, opening on trip** — reading "on" at rest and "off" when actually pinched, the reverse of every other input in the project. Missed, this would have made the safety interlock trigger constantly during normal operation while missing a real pinch entirely — about as bad as a safety bug gets. Fixed with a per-input polarity flag rather than assuming one global convention.

06 Firmware, and its bugs

The state machine grew from a simple two-phase cycle into something closer to the original board's actual behavior: `BOOT_HOMING → IDLE ⇄ CAT_PRESENT → WAIT_TIMER → CYCLE_TO_DUMP → CYCLE_DUMP_PAUSE → CYCLE_DUMP_SHAKE → CYCLE_TO_HOME → CYCLE_HOME_OVERSHOOT → CYCLE_HOME_SETTLE → IDLE`, plus `SAFETY_STOP` and `FAULT`. Real hardware testing found real bugs in it:

> THE MOTOR NEVER ACTUALLY REVERSED
>
> The first real MQTT-triggered cycle test: "it just kept turning until it found home and that caused it to jump over the stop for dump." The Dump→Home transition changed a state variable without ever stopping or reversing the motor — it sailed straight through Dump and only stopped by chance, finding Home again on the far side of a full rotation.

> HOMING OVERSHOT, EVEN AFTER THAT FIX
>
> `BOOT_HOMING` ran forward with no idea which direction Home actually was from an arbitrary starting position. Redesigned to seek Dump first — the same direction as a normal cycle's first leg — then reverse to Home from there. Deterministic regardless of starting position, and it reuses direction logic already proven correct rather than a new, unvalidated strategy.

The status LEDs went through two designs before landing on the right one: a bi-color (2-pin) LED faking amber by blinking red and green together, corrected once the actual hardware turned out to be three fully discrete LEDs on three separate GPIOs. The corrected version replicates the original board's documented light language exactly — solid green for standby, solid yellow while cycling, solid red for an interruption, flashing red for an overweight globe, flashing yellow for a detected pinch.

07 The dashboard

The ESP32 hosts its own web dashboard over LittleFS — no separate server, no app to install. It grew from a single status page into three tabs: live status and actions, a usage-analytics view (visit history, average time between visits split into day and night), and settings covering WiFi/MQTT/behavior plus browser-based firmware updates.

One recurring theme: anything involving *time* needed care. The board's clock is deliberately UTC-only (no DST math to get wrong) — which means "today" can never be computed on the device itself without knowing the viewer's timezone. Visit counts and day/night classification are computed client-side, in the browser, against each visit's local time — not trusted from the device's own rolling-window math, which quietly counted a visit from the previous evening as "today" depending on what hour it happened to be when the page loaded.

08 Chasing a WiFi ghost

The most stubborn bug so far wasn't in the litter-robot logic at all — it was WiFi refusing to associate, intermittently, logging nothing more informative than `ASSOC_LEAVE`. Each theory got tested and, mostly, ruled out rather than assumed:

- **Dirty resets left the network stack corrupted** — plausible, and a real fix (tearing down WiFi/lwIP after a brownout/watchdog/panic reset) — but it recurred on a genuinely clean power-on reset, which has nothing "dirty" to clean up. Not the (whole) answer.
- **Motor driver noise during homing** — the motor runs immediately on boot, independent of WiFi, so the timing lined up on paper. Directly checked against the status LED: no correlation. Ruled out.
- **Power supply** — tested on a buck converter and plain USB both; failed on both at different times, with no brownout message ever logged. Inconclusive at best.

Rather than guess a fourth mechanism blind, the response was to add instrumentation instead of another theory: signal strength exposed on the dashboard, reset cause printed on every boot (not just suspicious ones), WiFi connect attempts logged with timing, and — on the chance the home network has more than one access point sharing an SSID — the board now scans first and pins to whichever AP actually has the strongest signal, rather than whichever one it happens to see first.

> A SEPARATE, RELATED BUG FOUND ALONG THE WAY
>
> mDNS (`lr2redux.local`) turned out to initialize exactly once, ever — the first successful connect after boot — and never re-arm itself after a reconnect. Direct-IP access kept working the whole time, which is what made it look "sporadic" rather than broken: two features that don't share any state, one silently going stale while the other kept working.

The root cause of the original `ASSOC_LEAVE` loop is still open. This section will get a resolution once one actually lands.

09 Where it stands

Both firmware images (production and the standalone diagnostic tool) build clean, and the web dashboard builds and typechecks clean. The 7-pin harness connector is fully resolved and, as of the hall-sensor install, fully wired through the original connector rather than bench leads — nothing on it is provisional anymore. The weight/anti-pinch physical split is done and bench-verified.

What's still ahead: full motor/assembly installation, and a honest list of firmware behavior that builds clean but hasn't been proven on a complete, assembled unit yet — cycle-phase timing constants (dwell, shake, overshoot durations) are current best guesses pending a stopwatch, not measurements.

The rest of the story — final assembly, first full unattended cycle, the timing constants getting tuned for real — isn't written yet.
